

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления

Кафедра интеллектуальных информационных технологий

**Отчет по лабораторной работе №2
по дисциплине
«Аппаратное обеспечение интеллектуальных систем»**

Выполнил
студент группы 421702:

Захаренков И.Д.

Проверил:

Захаров В.В.

Минск 2026

Вариант 11

Функция:

$$\overline{(\overline{x_1} + x_2) \cdot x_2 \cdot \overline{x_3}}$$

Цель:

- повторение и закрепление материала по преобразованию логических функций,
- освоение навыков по использованию свойств логических функций, законов и следствий алгебры логики для преобразования логических функций представленных в разных формах.

Задачи:

Составить и проверить программу, обеспечивающую преобразование функции f_i (x_1, x_2, x_3), представленной в произвольной форме, в СДНФ и СКНФ, определение значений и запись функции f_i сднф(x_1, x_2, x_3) и f_i скнф(x_1, x_2, x_3) в числовой форме (по методу Мак-Класки).

Программа должна быть способна:

- записывать в память исходные функции (при этом программа должна уметь записывать одиночные аргументы с отрицаниями и без, групповые и общие отрицания);
- анализировать форму исходной функции и задание на ее преобразование;
- выполнять требуемое преобразование исходной функции;
- выводить результаты выполнения задания;

Результат программы:

[Таблица истинности]

№	x1	x2	x3	f29	Вес разряда
0	0	0	0	0	128
1	0	0	1	0	64
2	0	1	0	0	32
3	0	1	1	1	16
4	1	0	0	1	8
5	1	0	1	1	4
6	1	1	0	0	2
7	1	1	1	1	1

[Индекс функции]

i = 29 (двоичное: 00011101)

[СДНФ – Совершенная Дизъюнктивная Нормальная Форма]

$$\begin{aligned} \text{СДНФ}(x_1, x_2, x_3) &= \\ &= (\neg x_1 \wedge x_2 \wedge x_3) \vee (x_1 \wedge \neg x_2 \wedge \neg x_3) \vee (x_1 \wedge \neg x_2 \wedge x_3) \vee (x_1 \wedge x_2 \wedge x_3) \end{aligned}$$

[СКНФ – Совершенная Конъюнктивная Нормальная Форма]

$$\begin{aligned} \text{СКНФ}(x_1, x_2, x_3) &= \\ &= (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \neg x_3) \wedge (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3) \end{aligned}$$

[Числовая форма]

$$(x_1, x_2, x_3) = V(3, 4, 5, 7) = \Lambda(0, 1, 2, 6)$$

Вывод:

СКНФ (Совершенная конъюнктивная нормальная форма) - способ представления логической функции в виде конъюнкции элементарных дизъюнкций.

СДНФ (Совершенная дизъюнктивная нормальная форма) - способ представления логической функции в виде дизъюнкции элементарных конъюнкций.

Эти формы называются нормальными, потому что все члены функций в данном случае имеют вид элементарных (все элементы являются одиночными аргументами или отрицанием одиночных аргументов) дизъюнкций или конъюнкций. Они называются совершенными, потому что все члены этих выражений имеют высший ранг (кол-во аргументов в функции), т.е. являются конституентами.

Конституента единицы - элементарная конъюнкция, являющаяся функцией всех аргументов заданного набора.

Конституента нуля - элементарная дизъюнкция, являющаяся функцией всех аргументов заданного набора.

Произвольная форма является наиболее интуитивной для первоначальной записи функции, однако она не является уникальной, часто содержит избыточные элементы и неудобна для аппаратной реализации из-за сложности анализа.

СДНФ и СКНФ являются каноническими (единственными для данной функции) формами. Они однозначно строятся по таблице истинности (СДНФ по единицам, СКНФ по нулям). Однако эти формы являются громоздкими и неудобными для обработки при помощи ЭВМ.

Числовая форма (Мак-Класки) является наиболее краткой и удобной для обработки при помощи ЭВМ, поскольку использует только номера конституент. Зная числовой индекс функции и рассматривая его как $2n$ -разрядное двоичное число где n -количество аргументов функции, легко составить СДНФ или СКНФ

этой функции. Однако данный метод является менее наглядным, поскольку из записи не видна структура функции.

Приложение А

Листинг кода

```
from itertools import product as iproduct

def tokenize(expr: str) -> list[tuple[str, str]]:
    tokens: list[tuple[str, str]] = []

    expr = (expr
            .replace(' ', '')
            .replace('.', '*')
            .replace('•', '*')
            .replace('^', '*')
            .replace('v', '+')
            .replace('~', '!')
            .replace('¬', '!'))

    i = 0
    while i < len(expr):
        c = expr[i]

        if c == 'x' and i + 1 < len(expr) and expr[i + 1].isdigit():
            j = i + 1
            while j < len(expr) and expr[j].isdigit():
                j += 1
            tokens.append(('VAR', expr[i:j]))
            i = j

        elif c == '!':
            tokens.append(('NOT', '!'))
            i += 1

        elif c == '+':
            tokens.append(('OR', '+'))
            i += 1

        elif c in ('*', '&'):
            tokens.append(('AND', c))
```

```

elif c in ('*', '&'):
    tokens.append(('AND', c))
    i += 1

elif c == '(':
    tokens.append(('LPAREN', '('))
    i += 1

elif c == ')':
    tokens.append(('RPAREN', ')'))
    i += 1

else:
    raise SyntaxError(
        f"Неизвестный символ '{c}' в позиции {i}.\n"
        "Используйте x1..xN для переменных, ! для отрицания, "
        "+ для ИЛИ, * для И, () для группировки."
    )

return tokens

```

class Parser:

```

def __init__(self, tokens: list[tuple[str, str]]):
    self.tokens = tokens
    self.pos = 0

def peek(self) -> tuple[str, str] | None:
    return self.tokens[self.pos] if self.pos < len(self.tokens) else None

def consume(self, expected: str | None = None) -> tuple[str, str]:
    tok = self.tokens[self.pos]
    if expected and tok[0] != expected:
        raise SyntaxError(f"Ожидался '{expected}', получен '{tok[1]}'")
    self.pos += 1
    return tok

def _can_start_factor(self) -> bool:
    tok = self.peek()
    return tok is not None and tok[0] in ('NOT', 'VAR', 'LPAREN')

def parse(self):
    node = self.parse_expr()
    if self.peek() is not None:
        raise SyntaxError(f"Неожиданный токен '{self.peek()[1]}' в конце выражения")
    return node

def parse_expr(self):
    left = self.parse_term()
    while self.peek() and self.peek()[0] == 'OR':
        self.consume('OR')
        right = self.parse_term()
        left = ('OR', left, right)
    return left

```

```

def parse_term(self):
    left = self.parse_factor()
    while True:
        tok = self.peak()
        if tok is None:
            break
        if tok[0] == 'AND':
            self.consume('AND')
            left = ('AND', left, self.parse_factor())
        elif self._can_start_factor():
            left = ('AND', left, self.parse_factor())
        else:
            break
    return left

def parse_factor(self):
    if self.peak() and self.peak()[0] == 'NOT':
        self.consume('NOT')
        return ('NOT', self.parse_factor())
    return self.parse_atom()

def parse_atom(self):
    tok = self.peak()
    if tok is None:
        raise SyntaxError("Неожиданный конец выражения")
    if tok[0] == 'LPAREN':
        self.consume('LPAREN')
        node = self.parse_expr()
        if self.peak() is None or self.peak()[0] != 'RPAREN':
            raise SyntaxError("Отсутствует закрывающая скобка ')"")
        self.consume('RPAREN')
        return node
    if tok[0] == 'VAR':
        self.consume('VAR')
        return ('VAR', tok[1])
    raise SyntaxError(f"Неожиданный токен '{tok[1]}")

```

```

def parse_expression(expr: str):
    tokens = tokenize(expr)
    if not tokens:
        raise ValueError("Пустое выражение")
    return Parser(tokens).parse()

```

```
def evaluate(node, assignment: dict[str, int]) -> int:
    kind = node[0]
    if kind == 'VAR':
        name = node[1]
        if name not in assignment:
            raise ValueError(f"Переменная '{name}' отсутствует в наборе")
        return assignment[name]
    if kind == 'NOT':
        return 1 - evaluate(node[1], assignment)
    if kind == 'AND':
        return evaluate(node[1], assignment) & evaluate(node[2], assignment)
    if kind == 'OR':
        return evaluate(node[1], assignment) | evaluate(node[2], assignment)
    raise ValueError(f"Неизвестный тип узла АСД: {kind}")
```

```
def collect_variables(node) -> set[str]:
    kind = node[0]
    if kind == 'VAR':
        return {node[1]}
    if kind == 'NOT':
        return collect_variables(node[1])
    if kind in ('AND', 'OR'):
        return collect_variables(node[1]) | collect_variables(node[2])
    return set()
```

```
def build_truth_table(node, variables: list[str]) -> list[tuple[tuple, int]]:
    table: list[tuple[tuple, int]] = []
    for values in iproduct([0, 1], repeat=len(variables)):
        assignment = dict(zip(variables, values))
        result = evaluate(node, assignment)
        table.append((values, result))
    return table
```

```
def _fmt_conjunction(values, variables: list[str]) -> str:
    parts = [f'¬{v}' if val == 0 else v for v, val in zip(variables, values)]
    return '(' + ' ∧ '.join(parts) + ')'
```

```
def _fmt_disjunction(values, variables: list[str]) -> str:
    parts = [f'¬{v}' if val == 1 else v for v, val in zip(variables, values)]
    return '(' + ' ∨ '.join(parts) + ')'
```



```

def compute_sdnf(truth_table, variables: list[str]) -> tuple[list[int], list[str]]:
    minterms, terms = [], []
    for values, result in truth_table:
        if result == 1:
            minterms.append(int(''.join(map(str, values)), 2))
            terms.append(_fmt_conjunction(values, variables))
    return minterms, terms

def compute_sknf(truth_table, variables: list[str]) -> tuple[list[int], list[str]]:
    maxterms, terms = [], []
    for values, result in truth_table:
        if result == 0:
            maxterms.append(int(''.join(map(str, values)), 2))
            terms.append(_fmt_disjunction(values, variables))
    return maxterms, terms

def compute_index(truth_table) -> int:
    return int(''.join(str(r) for _, r in truth_table), 2)

def print_truth_table(truth_table, variables: list[str], index: int) -> None:
    col_w = max(len(v) for v in variables) + 2
    f_label = f'{index}'

    header = ' № | ' + ' | '.join(f'{v:^{col_w}}' for v in variables) + ' | ' + f_label + f'^{max(3, len(f_label))}' + ' | Вес разряда'
    sep = '-' * len(header)
    print(sep)
    print(header)
    print(sep)

    num_rows = len(truth_table)

    for j, (values, result) in enumerate(truth_table):
        weight = 2 ** (num_rows - 1 - j)

        row = f' {j:>3} | '
        row += ' | '.join(f'{v:^{col_w}}' for v in values)
        row += f' | {result:^{max(3, len(f_label))}} | {weight:^11}'
        print(row)
    print(sep)

def print_results(
    expr_input: str,
    variables: list[str],
    truth_table,
    sdnf_terms: list[str],
    sknf_terms: list[str],
    minterms: list[int],
    maxterms: list[int],
    index: int
) -> None:
    func_name = f'F{index}'
    var_str = ', '.join(variables)

    print('\n' + '=' * 64)
    print(f' Исходная функция : {expr_input}')
    print(f' Переменные : {var_str} (n = {len(variables)})')
    print('=' * 64)

    print('\n[ Таблица истинности ]')
    print_truth_table(truth_table, variables, index)

```



```

print(f'\n[ Индекс функции ]')
print(f' i = {index} (двоичное: {index:0{2 ** len(variables)}b})')

print(f'\n[ СДНФ – Совершенная Дизъюнктивная Нормальная Форма ]')
if sdnf_terms:
    print(f' СДНФ({var_str}) =')
    chunk = 4
    for i in range(0, len(sdnf_terms), chunk):
        prefix = ' = ' if i == 0 else ' '
        suffix = ' ' if i + chunk < len(sdnf_terms) else ''
        print(prefix + ' '.join(sdnf_terms[i:i + chunk]) + suffix)
else:
    print(f' СДНФ = 0 (функция тождественно ЛОЖНА)')

print(f'\n[ СКНФ – Совершенная Конъюнктивная Нормальная Форма ]')
if sknf_terms:
    print(f' СКНФ({var_str}) =')
    chunk = 4
    for i in range(0, len(sknf_terms), chunk):
        prefix = ' = ' if i == 0 else ' '
        suffix = ' ^ ' if i + chunk < len(sknf_terms) else ''
        print(prefix + ' ^ '.join(sknf_terms[i:i + chunk]) + suffix)
else:
    print(f' СКНФ = 1 (функция тождественно ИСТИННА)')

print(f'\n[ Числовая форма ]')
v_part = 'V(' + ', '.join(map(str, minterms)) + ')' if minterms else 'V(∅)'
l_part = 'Λ(' + ', '.join(map(str, maxterms)) + ')' if maxterms else 'Λ(∅)'
print(f' ({var_str}) = {v_part} = {l_part}')
print()

```

HELP_TEXT = """

Синтаксис ввода логической функции

Переменные : x1, x2, x3 (или xN для любого N)
 Отрицание : !x1 или !(выражение)
 И (AND) : * или & или пробел (неявное И)
 ИЛИ (OR) : +
 Скобки : ()

Примеры вариантов из лабораторной работы:

Вар.02: $!(!x1 + !x2) * !x1 * x3$
 Вар.06: $!(x1 + !x2) * !(!x1 * x3)$ ← общее отрицание
 Вар.10: $!(!x1 + !x2) * !x2 * x3$
 Вар.11: $!(!x1 + x2) * !(x2 * !x3)$
 Вар.20: $(x2 + x3) * !x1 * x3$

```

def main() -> None:
    print('=' * 64)
    print('    Лабораторная работа №2')
    print('    Преобразование логических функций в СДНФ и СКНФ')
    print('=' * 64)
    print(HELP_TEXT)

    while True:
        try:
            raw = input('Введите функцию (или "exit" для выхода):\n> ').strip()
        except (EOFError, KeyboardInterrupt):
            print('\nВыход.')
            break

        if not raw:
            continue
        if raw.lower() in ('exit', 'quit', 'выход', 'q'):
            print('Выход.')
            break

        try:
            ast = parse_expression(raw)
        except (SyntaxError, ValueError) as e:
            print(f'\n[Ошибка парсинга] {e}\n')
            continue

        vars_set = collect_variables(ast)
        if not vars_set:
            print('[Ошибка] Переменные не найдены.\n')
            continue

        variables = sorted(vars_set, key=lambda s: int(s[1:]))

        truth_table = build_truth_table(ast, variables)
        minterms, sdnf_terms = compute_sdnf(truth_table,
                                             variables)
        maxterms, sknf_terms = compute_sknf(truth_table,
                                              variables)
        index = compute_index(truth_table)

        print_results(
            expr_input=raw,
            variables=variables,
            truth_table=truth_table,
            sdnf_terms=sdnf_terms,
            sknf_terms=sknf_terms,
            minterms=minterms,
            maxterms=maxterms,
            index=index,
        )

if __name__ == '__main__':
    main()

```